

SENTINEL

HARDWARE MONITOR

Zertifizierung 2025

Software Developer Kurs 3380 (IHK)

Dokumentation zur Projektarbeit

SENTINEL Hardware-Monitor in Python

Software zur Überwachung von Hardware-Daten

Abgabedatum: 20.07.2025

Prüfungsbewerber:

Florian Schimbold

Schloßstr. 16

76646 Bruchsal

Inhaltsverzeichnis

1 Einleitung ... 1
1.1 Projektbeschreibung ... 1
1.2 Projektziel ... 1
1.3 Projektbegründung ... 1
1.4 Projektschnittstellen ... 2
2 Projektplanung ... 2
2.1 Projektphasen ... 2
2.2 Abweichungen vom Projektantrag ... 3
2.3 Ressourcenplanung ... 3
2.4 Entwicklungsprozess ... 3
3 Analysephase ... 4
3.1 Ist-Analyse ... 4
3.1.1 Make or Buy-Entscheidung ... 4
3.2 Qualitätsanforderungen ... 4
3.3 Fachkonzept ... 5
4 Entwurfsphase ... 5
4.1 Zielplattform ... 5
4.2 Architekturdesign ... 6
4.3 Entwurf der Benutzeroberfläche ... 6
4.4 Geschäftslogik ... 6
4.5 Maßnahmen zur Qualitätssicherung ... 6
5 Implementierungsphase ... 6
5.1 Implementierung der Datenstrukturen ... 6
5.2 Implementierung der Benutzeroberfläche ... 7
5.3 Implementierung der Geschäftslogik ... 7
6 Abnahmephase ... 10
7 Einführungsphase ... 10
8 Dokumentation ... 11
9 Fazit ... 11
9.1 Soll-/Ist-Vergleich ... 11
9.2 Lessons Learned ... 12
9.3 Ausblick ... 12
Anhang ...

Abkürzungsverzeichnis

API
: Application Programming Interface

CPU
: Central Processing Unit

GPU
: Graphics Processing Unit

GUI
: Graphical User Interface

HDD
: Hard Disk Drive

MVC
: Model View Controller

OSD
: On Screen Display

RAM
: Random Access Memory

SQL
: Structured Query Language

SSD
: Solid State Drive

1 Einleitung

Die folgende Projektdokumentation beschreibt die Planung, Entwicklung und Implementierung von „SENTINEL“, einem Hardware-Überwachungstool, dass als Abschlussprojekt im Rahmen des Kurses (3380) zur Erhaltung des gleichnamigen Zertifikats „Software Developer (IHK)“ realisiert wurde.

1.1 Projektbeschreibung

In vielen Bereichen der Informationstechnik, wie auch im privaten, ist es unabdingbar sein technisches Arbeitsumfeld zu überwachen. Anspruchsvolle Anwendungen erfordern oft ein hohes Maß an Rechenleistung und somit eine moderne und leistungsfähige Hardware. Diese gilt es zu überwachen, um kritische Ausfälle der Hardware und deren teils fatale Folgen zu vermeiden.
Dieses Überwachungstool ermöglicht es dem Anwender, alle Hardware relevanten Datensätze über eine grafische Benutzeroberfläche einzusehen und zu überwachen. So kann dieser Hardware-Monitor zur Systemdiagnose, Leistungsoptimierung und auch zur potenziellen Fehlererkennung eingesetzt werden.

1.2 Projektziel

Das Ziel dieses Projekts ist die Realisierung einer Hardware-Überwachungssoftware, die relevante Systemparameter wie CPU und GPU-Auslastung, Temperatur, Arbeitsspeichernutzung, Festplattenaktivität und Lüftergeschwindigkeiten in Echtzeit überwacht und visualisiert. Die Erfassung der Hardware-Kennzahlen erfolgt durch die direkte Abfrage der Sensoren und Systemkomponenten.

Eine benutzerfreundliche GUI ist wichtig für eine übersichtliche Darstellung der Daten.Die Benutzeroberfläche soll intuitiv und vor allem übersichtlich sein, somit wird eine effiziente Analyse der Daten sichergestellt. Zusätzlich ist die Option eines Overlay als On-Screen-Display (OSD) vorgesehen, der über der aktuellen Bildschirmausgabe dem Anwender die von ihm ausgesuchten Parameter als Information zur Verfügung stellt.

Ein Echtzeitdiagramm der verarbeiteten Daten der Komponenten soll helfen sich ein besseres und übersichtlicheres Bild der Lage zu verschaffen. Außerdem sollen Leistungsdaten über einen längeren Zeitraum aufgezeichnet werden um diese bei Bedarf auszuwerten. Diese Daten sollen in einer CSV-Datei oder in einer SQL-Datenbank gespeichert und als Langzeitdiagramm dargestellt werden.

1.3 Projektbegründung

Aufgrund der Teilnahme an dem Kurs „Software Developer (IHK), habe ich ein für mich passendes Thema zur abschließenden Projektprüfung zur Zertifizierung zum „Software Developer (IHK)“ gesucht.

Die Wahl fiel auf einen Hardware-Monitor, weil dieser in meinem privaten Umfeld fast täglich genutzt wird und die Programmierung dessen von ungemeinem Interesse für mich ist. Es ist nicht allein als Projektarbeit für das Zertifikat zu sehen, sondern auch

als zukünftige Basis für mehr Ideen, Erweiterungen und Implementierungen. Außerdem wird es einen echten Mehrwert haben, da es kein rein theoretisches Beispiel ist, sondern ein konkret einsetzbares Tool. Es gibt natürlich reichlich Software auf dem Markt, die als Hardware-Monitor fungieren, allerdings habe ich einige ergänzende Ideen, die ich für mich einbinden möchte, jetzt sowie in Zukunft.

Gemeint sind Visualisierungen in Echt- und Langzeitdiagrammen, Benachrichtigungen bei kritischen Werten (Schwellwertüberwachung), zusätzliche Tools wie Benchmarks und eventuelle API Einbindungen auf Webservern, beispielsweise für CPU Rankings. Die Möglichkeiten sind reichlich, wie auch der angestrebte Lernprozess.

1.4 Projektschnittstellen

Für den Einsatz und Anwendungszweck von „SENTINEL“ ist eine GUI als Schnittstelle via tkinter geplant.
Dies ermöglicht einen zweckmäßig vereinfachten Umgang mit der Software.
Außerdem möchte ich weitestgehend modularisieren, das bedeutet das importierte Module in diesem Sinn auch Schnittstellen darstellen.

Da dieses Projekt der Zertifizierung dient, bin vorerst ich der einzige Anwender. Zukünftige potenzielle Anwender sind Freunde und Bekannte.

2 Projektplanung

2.1 Projektphasen

Für die Realisierung des Projekts wurde ein Zeitrahmen von 80 Stunden festgelegt. Dieser durfte um maximal 20 % unterschritten werden. Start des Projekts war der 02.06.2025. Abgabe der Projektarbeit ist der 20.07.2025.

Die Zeitplanung sowie die dazugehörigen Hauptphasen lassen sich in der folgenden Tabelle entnehmen. Eine detailliertere Übersicht der Zeitplanung befindet sich im Anhang A.1: Detaillierte Zeitplanung

Projektphase	Geplante Zeit
Analyse	3h
Entwurf	3h
Implementierung	35h
Abnahme	2h
Einführung	2h
Dokumentation	35h
Gesamt	80h

2.2 Abweichungen vom Projektantrag

Eine Abweichung von meinem Projektantrag gab es nicht.

2.3 Ressourcenplanung

Für die Umsetzung des Projekts wurde sich für verschiedene Open-Source Lösungen entschieden. Die verwendete integrierte Entwicklungsumgebung ist die PyCharm Community Edition von JetBrains. Im Rahmen des Kurses lernten wir den Umgang mit Thonny, aber auch mit PyCharm. Pycharm ist für mich intuitiver und effizienter im Umgang. Dies erforderte allerdings etwas Einarbeitung.

Für die Gestaltung der Benutzeroberfläche wurde mit TkInter gearbeitet. Anfangs gab es die Überlegung mit PyQt6 zu arbeiten, allerdings wurde diese Idee aufgrund immenser Einarbeitung zur Seite gelegt.

Es folgt eine Liste mit allen Ressourcen die geplant und auch tatsächlich für das Projekt eingesetzt wurden. Die Liste besteht aus dem Arbeitsumfeld, verwendete Hard- und Software, sowie genauen Hardware Spezifikationen um eine vereinfachte Nachvollziehbarkeit in der späteren Dokumentation zu gewährleisten.

Arbeitsumfeld

Heimbüro mit PC, Peripheriegeräte wie Maus, Tastatur und Bildschirm

Hardware

Mainboard:	Gigabyte X670 X AX V2
CPU:	AMD Ryzen 9 7900X3D
GPU:	AMD Radeon RX 7700 XT
RAM:	32 GB
SSD:	SAMSUNG 990 Pro 1 TB

Software

Betriebssystem: Windows 11 Pro 64-bit
IDE: PyCharm Community Edition

2.4 Entwicklungsprozess

Für meine Projektprüfung habe ich mich bewusst für das Wasserfallmodell als Entwicklungsprozess entschieden. Dieses folgt einer klaren, linearen Phasenabfolge (Analyse, Entwurf, Implementierung, Test, Einsatz) Diese Struktur hilft ungemein das Projekt systematisch zu planen und Schritt für Schritt umzusetzen.

Für eine kommerzielle Entwicklung hätte ich mich allerdings für agilere Entwicklungsmethoden entschieden, da ein Hardware Monitor oft ändernden Anforderungen gegenübersteht. (z. B. Kompatibilität verschiedener CPUs/GPUs, Sensoren, allgemein Hardware verschiedener Hersteller sowie Treiber etc.

3 Analysephase

3.1 Ist-Analyse

In der Analyse habe ich festgestellt dass reichlich Software der gleichen Art auf dem offenen Markt zur Verfügung steht. Aus dem Selbstnutzen solcher Software und der Teilhabe des Kurses wuchs das Interesse, einen eigenen Hardware Monitor zu entwickeln.

Geplant ist es ein Programm zu erstellen in dem ich alle relevanten Parameter von verbauter Hardware eines PC – Systems auslesen und überwachen kann. Dies beinhaltet allgemeine Informationen wie Hersteller, Modell und Seriennummern. Außerdem dynamische Parameter wie die Auslastungen und Taktraten, Lese- und Schreibgeschwindigkeiten, Spannungen und Stromverbrauch, Temperaturabfragen der Sensoren der verschiedenen Komponenten.

Dynamische Werte sollen als Virtualisierung mit Echtzeitdiagrammen dargestellt werden. Außerdem soll ein On-Screen-Display als Überwachungstool dienen, damit man bei leistungsintensiven Anwendungen oder Videospielen einen konstanten Überblick seiner Hardware und deren Auslastung behält. Damit kann man eventuellen Überlastungen seines Systems entgegenwirken und wenn nötig, Änderungen und Drosselungen vornehmen.

Die geplant auszulesende Hardware umfasst unter anderem das Mainboard, den Hauptprozessor, die Grafikkarte(n), den Arbeitsspeicher, sowie den Datenspeicher.

Zur Visualisierung soll eine Benutzeroberfläche dienen, die intuitiv und übersichtlich sein soll. Damit wird gewährleistet, wichtige Parameter effizient abrufen und ablesen zu können.

Ziel ist es eine lauffähige .exe zu erzeugen.

3.1.1 Make or Buy-Entscheidung

Wie schon in der Ist-Analyse festgestellt, gibt es einige fertige Produkte auf dem Markt die das geplante Spektrum abdecken. Ich habe allerdings noch viele Ideen die ich gerne umsetzen und ausprobieren möchte, darum entschied ich mich für eine eigene Lösung.

3.2 Qualitätsanforderungen

Die Software soll wartbar sein. Dies bedeutet unter anderem einen einfach und effizient gehaltenen Code, mit einer klaren und übersichtlichen Gestaltung. Somit ist zukünftig eine vereinfachte potenzielle Fehlerbehebung gewährleistet.

Natürlich soll die Software auch fehlerfrei und robust sein

Die Benutzbarkeit (Usability) soll intuitiv und die Benutzeroberfläche leicht zu bedienen sein.

Außerdem setze ich eine gute Laufzeit Performance voraus, so wäre ein träger Umgang mit der Benutzeroberfläche nicht zielführend.

3.3 Fachkonzept

Geplante Funktionen des Programms:

Allgemeine Hardware-Informationen auslesen:
Hersteller, Modell und Seriennummern aller Komponenten

Analysephase

Dynamische Parameter überwachen:
Auslastung und Taktraten von CPU/GPU
Lese- und Schreibgeschwindigkeiten des Datenspeichers
Spannungen und Stromverbrauch
Temperaturen der verschiedenen Sensor-Schnittstellen

Unterstützte Hardware-Kategorien:
Mainboard
Hauptprozessor (CPU)
Grafikkarte(n) (GPU)
Arbeitsspeicher (RAM)
Datenspeicher (HDD/SSD)

Echtzeit-Visualisierung:
Darstellung der dynamischen Werte in interaktiven, aktualisierten Diagrammen

Datenbankspeicherung
Ablage in einer Datenbank in Verbindung mit Langzeitdiagrammen

On-Screen-Display (OSD):
Einblendung der wichtigsten Monitoring-Daten während leistungsintensiver Anwendungen oder Spielen

Schutzmechanismen:
Warnungen bei drohender Systemüberlastung

Benutzeroberfläche:
Intuitiv und übersichtlich, um Parameter schnell abrufen und ablesen zu können

4 Entwurfsphase

4.1 Zielplattform

Ziel ist es, eine ausführbare Datei vom Typ .exe für Windows 11 bereitzustellen. Da ich mit diesem Betriebssystem arbeite, soll dieses Programm auch für dieses entwickelt werden. Die Programmiersprache dafür ist Python. Ich habe mich für Python entschieden aufgrund der Hardwarenähe und vor allem weil es Schwerpunkt des Kurses war. Prinzipiell eignen sich aber auch andere Programmiersprachen dazu, wie z. B. CPP. Da die Abfrage der Daten allgemein und nicht Hersteller abhängig gehalten werden soll spielt die vorhandene Hardware eine untergeordnete Rolle.

4.2 Architekturdesign

Für mein Projekt habe ich bewusst das Model-View-Controller-Pattern verwendet, um eine klare Trennung von Zuständigkeiten innerhalb der Anwendung sicherzustellen. Diese Struktur ermöglicht es, die grafische Benutzeroberfläche (View), die Logik zur Datenerhebung (Model) und die Steuerung der Anwendung (Controller) logisch und funktional voneinander zu trennen. Die Vorteile dieser Architektur äußern sich durch eine einfache Erweiterbarkeit. Außerdem lassen sich Fehler gezielt im jeweiligen Modul suchen und beheben. Ein auf mein Hauptfenster angewandest Beispiel für ein MVC-Pattern als ASCII-Diagramm ist im Anhang zu sehn.

4.3 Entwurf der Benutzeroberfläche

Die Benutzeroberfläche wird aufgrund leichter Handhabe und weil es schon in Python integriert ist, mit TkInter erstellt. Die Anwendung besteht aus einem Hauptfenster, das nicht größenänderbar sein soll. Auf der linken Seite ist ein schmales Menü mit Buttons(oben angeheftet). Auf der rechten Seite (ca.80%) sollen die Inhalte bei Button Klick in verschiedenen Frames dargestellt werden. Wenn man das Programm startet, kommt man auf die Startseite und sieht das selbst erstellte Logo, dies stellt gleichzeitig den ersten Menüpunkt (Startseite) da. Unter Startseite folgend kommen die Buttons für: Mainboard, Hauptprozessor, Grafikkarte(n), Datenspeicher und Arbeitsspeicher. Bei Button Klick öffnet sich auf der rechten Seite des Fensters jeweils ein Frame mit den jeweiligen statischen und/oder dynamischen Daten. Ganz unten angeheftet im Menü soll ein „beenden“ sein der das Programm beendet. Darüber noch ein Informationsbutton, der bei Button Klick allgemeine Informationen über das Projekt wieder in einem Frame auf der rechten Seite anzeigt.
Die ganze Benutzeroberfläche soll in dunklen Grautönen und weißer Schrift erfolgen. Somit ist ein guter Kontrast und eine vernünftige Lesbarkeit gewährleistet.

4.4 Geschäftslogik

Zur Verdeutlichung der Geschäftslogik wurde ein Klassendiagramm vom Sentinel Hauptfenster erstellt. Das Diagramm befindet sich im Anhang.

4.5 Maßnahmen zur Qualitätssicherung

Um die Qualität der Projektarbeit zu gewährleisten wird geplant mehrere Schreibtischtests durchzuführen.

Während der Implementierung muss häufig getestet werden ob die Module und deren Import ins Hauptprogramm korrekt ausgeführt werden.

Zusätzlich sind mehrere Unit-Tests geplant. In welchem Umfang zeigt sich erst nach der Implementierung.

Auch wurde der Code permanent überarbeitet, damit dieser sauber und deutlich zu lesen ist.

5 Implementierungsphase

5.1 Implementierung der Datenstrukturen

In meinem Projekt habe ich keine relevanten Datenstrukturen wie z.B. Datenbanken oder ähnliches.

5.2 Implementierung der Benutzeroberfläche

Für die Implementierung der Benutzeroberfläche importiere ich in meinem Hauptprogramm tkinter und lege eine Klasse Hauptfenster an, in deren Konstruktor ich das Design des Fensters festlege. Den Titel mit self.title("…"), die Hintergrundfarbe über self.configure(bg="…"), die Fenstergröße mittels self.geometry("Breite×Höhe") und die Einschränkung der Größenänderung durch self.resizable(width=0, height=0). Danach definiere ich lokale Farbvariablen wie farbe_hintergrund_menue = "gray8" für Sidebar, Container und die darin angezeigten

Implementierungsphase

Frames. Sidebar und Container werden jeweils als tk.Frame-Widgets erzeugt und mit .pack() an den linken beziehungsweise rechten Fensterrand platziert.

Für die eigentlichen Seiten-Frames initialisiere ich ein Dictionary mit self.frames = {}, erzeuge dann für jede Frame-Klasse (z. B. Startseite, CPU usw.) eine Instanz, speichere sie unter ihrem Klassennamen im Dictionary und ordne sie alle per grid(row=0, column=0, sticky="nsew") übereinander im Container an. So sind alle Frames sofort angelegt und belegen bei Bedarf automatisch den gesamten Containerbereich, während ich sie später einfach per tkraise() in den Vordergrund holen kann.

Der Code legt einmalig alle Anwendungs-Seiten als tk.Frame-Instanzen an, speichert sie im Dictionary (self.frames) unter ihrem Klassennamen und stapelt sie übereinander im gleichen Feld des zentralen Container-Widgets.

Anschließend wird das Grid des Containers so konfiguriert, dass die einzelne Zeile und Spalte beim Ändern der Fenstergröße mitwachsen. So sind alle Frames sofort verfügbar und belegen stets den gesamten Bereich des Containers, ohne dass sie bei der Erzeugung bereits sichtbar sein müssen. Später kann man dann einfach per tkraise() den gewünschten Frame in den Vordergrund holen, um zwischen den Seiten zu wechseln.

Danach wird zunächst in einer Schleife für jeden Menüpunkt in der Liste ein tk.Button in der linken Sidebar erzeugt. Der Parameter text legt die Beschriftung fest (z. B. „Startseite“ oder „Grafikkarten(GPU)“), und über das command-Lambda wird beim Klicken jeweils self.erstelle_frame(...) mit dem entsprechenden Frame-Namen aufgerufen. Klickt man auf „CPU“, wird also erstelle_frame("CPU") ausgeführt und der CPU-Frame eingeblendet.

Anschließend wird außerhalb der Schleife ein zusätzlicher Button „Beenden“ angelegt, dessen command=self.destroy das Fenster schließt, sowie ein Button „Über Sentinel“, der per erstelle_frame("Information") die Info-Seite anzeigt. Mit dem abschließenden Aufruf self.erstelle_frame("Startseite") wird zum Programmstart die Startseite aktiviert, sodass direkt nach dem Öffnen des Hauptfensters der erste Frame sichtbar ist.

Mit der Bedingung if __name__ == "__main__": wird geprüft, ob das Skript direkt ausgeführt wird. Innerhalb dieses Blocks wird dann eine Instanz deiner Hauptfenster-Klasse erstellt (main_fenster = Hauptfenster()) und mit main_fenster.mainloop() die Ereignisschleife des Fensters gestartet. Dadurch öffnet sich die Benutzeroberfläche.

5.3 Implementierung der Geschäftslogik

Zunächst werden im Hauptprogramm die einzelnen Frame-Klassen definiert. Diese erben alle von tk.Frame und werden jeweils mit dem übergeordneten Container, einem Controller-Verweis sowie Hintergrund- und Schriftfarben initialisiert. In der Startseite wird beim Erzeugen das Sentinel-Logo aus einer Bilddatei geladen, in ein PhotoImage umgewandelt und schließlich in einem Label mittig dargestellt. Die Klassen Mainboard, CPU, GPU, Datenspeicher und Arbeitsspeicher folgen demselben Muster: Zuerst wird eine Überschrift mit einer fett formatierten Helvetica-Schriftgröße 24 als Label mit entsprechenden Farben angezeigt, danach wird jeweils ein spezielles Modul-Widget (z. B. ModulMainboardBios, ModulCpu usw.) als Instanzvariable angelegt, mit pack(fill="both", expand=True, …) in den Frame eingesetzt und so skaliert, dass es den verbleibenden Platz komplett einnimmt. In der Klasse Informati-

Implementierungsphase

on verhält es sich ebenso, hier trägt das Überschrift-Label den Text „Über Sentinel“ und im Anschluss wird das ModulInformation-Widget angezeigt. Dadurch sind alle Seiten als eigene Frames gekapselt und lassen sich über die zentrale Steuerung des Hauptfensters bei Bedarf in den Vordergrund holen.
Es folgt die Beschreibung der einzelnen Module, angelegt als eigenständige Programme, die wie zuletzt beschrieben in das Hauptprogramm integriert werden.

Mainboard/BIOS:
Die Klasse ModulMainboardBios erbt von tk.Frame und zeigt auf Knopfdruck alle Mainboard- und BIOS-Informationen eines Windows-Systems an. Im Konstruktor (__init__) wird zunächst über die private Methode _ausgabe_mainboard_bios_info() eine formatierte Zeichenkette mit Herstellernamen, Produktbezeichnung, Seriennummer, BIOS-Version und Release-Datum erzeugt. Diese Zeichenkette wird anschließend in einem tk.Label mit fester Monospace-Schrift (Consolas, Größe 18) dargestellt. der Text wird dabei linksbündig ausgerichtet, umgebrochen bei 1.000 Pixel Breite und mit den übergebenen Hintergrund- und Schriftfarben versehen. Mittels pack(expand=True, fill="both") füllt das Label automatisch den gesamten zur Verfügung stehenden Platz im Frame aus.

Die Methode _ausgabe_mainboard_bios_info() verwendet das wmi-Modul, um zunächst alle Einträge aus der WMI-Klasse Win32_BaseBoard auszulesen und die wichtigsten Felder in eine Liste von Textzeilen zu schreiben. Anschließend fügt sie eine Trennungslinie für die BIOS-Daten ein und liest aus Win32_BIOS alle relevanten Informationen aus. Am Ende werden alle gesammelten Zeilen mit Zeilenumbrüchen zu einem einzigen String zusammengefügt.

Im if __name__ == "__main__": Block wird das Modul auch als eigenständiges Programm lauffähig: Ein Hauptfenster (tk.Tk()) mit Titel „Mainboard/BIOS“ und Größe 800×600 Pixel wird erzeugt, das ModulMainboardBios-Frame dort hineingepackt und mit mainloop() die tkinter-Ereignisschleife gestartet, sodass die Hardware-Informationen bei Ausführung sofort angezeigt werden. So bleibt der Code gut Wartbar und das Testing kann modular betrieben werden. Die folgenden Module folgen genau diesem Muster.

Hauptprozessor(CPU):
Die Klasse ModulCpu erbt von tk.Frame und stellt in einem scrollbaren Textfeld alle wichtigen Prozessor-Informationen dar. Über das WMI-Modul wird eine Verbindung zum Namensraum root\CIMV2 aufgebaut, um später statische CPU-Daten auszulesen. Die Methode ausgabe_cpu_info() fragt dann per WMI die Basisinfo (Hersteller, Modell, physische und logische Kerne, Maximaltakt) ab und ergänzt mit dem psutil-Modul die aktuelle Taktfrequenz, falls verfügbar.

In update_info() werden diese statischen Werte mit dynamischen Daten kombiniert: psutil.cpu_percent() liefert die Auslastung aller Kerne, aus denen eine Gesamtauslastung berechnet wird. Alle Daten werden zu einer formatieren Liste von Textzeilen zusammengefügt, ins Textwidget geschrieben und sofort wieder auf „schreibgeschützt“ gesetzt. Schließlich sorgt self.after(self.aktualisierungsrate, self.update_info) dafür, dass sich die Anzeige im voreingestellten Intervall automatisch aktualisiert.

==== if __name__ == "__main__": Block ====

Grafikkarte(n):

Implementierungsphase

Die Klasse ModulGpu erbt von tk.Frame und zeigt in einem einzigen, großen tk.Label alle Grafikkarten-Informationen an, die sie in regelmäßigen Abständen über WMI abfragt. Im Konstruktor wird zuerst das Label, linker Ausrichtung und den übergebenen Hintergrund- und Schriftfarben angelegt und so gepackt, dass es den gesamten Frame ausfüllt. Zwei WMI-Verbindungen (statisch und dynamisch) werden eingerichtet, um einerseits feste Daten wie Modellname, Gesamtspeicher und Treiberversion zu lesen und andererseits Performance-Daten wie 3D-Auslastung und aktuellen Speichereinsatz zu sammeln.

Die Methode ausgabe_gpu_info_mit_update() durchläuft alle in Win32_VideoController gefundenen GPU-Objekte, konvertiert den Adapter-RAM in Megabyte, ermittelt aus den Performance-Counters die Auslastungswerte und den aktuell belegten Speicher, baut daraus eine formatierte Textliste auf und schreibt sie in das Label. Am Ende ruft sie sich selbst nach dem definierten Intervall per self.after() erneut auf, um die Anzeige fortlaufend zu aktualisieren.

==== if __name__ == "__main__": Block ====

Datenspeicher:

Die Klasse ModulDatenspeicher erbt von tk.Frame und präsentiert sowohl statische Festplatten- und SSD-Details als auch einen optionalen Geschwindigkeitstest in einem Label. Im Konstruktor wird zunächst eine StringVar angelegt, die als Textquelle für ein tk.Label dient, und ein Button „Geschwindigkeit testen“ erstellt, der beim Klick die Methode geschwindigkeitstest_datenspeicher() auslöst. Die Methode ausgabe_datenspeicher_info() liest über WMI alle Datenträger aus (Win32_DiskDrive) und sammelt Modell, Hersteller, Seriennummer, Firmware, Schnittstelle und Größe in einer Liste von Zeilen, die in der StringVar dargestellt wird.

Der Geschwindigkeitstest schreibt danach eine temporäre Datei von 100 MB in 1 MB-Blöcken, misst mit time.perf_counter() die Dauer und berechnet so die Schreibgeschwindigkeit in MB/s. Anschließend liest er die Datei blockweise zurück, bestimmt die Lesegeschwindigkeit und entfernt die Testdatei wieder. Beide Werte werden ans Ende der bestehenden Info angehängt und die StringVar entsprechend aktualisiert, sodass das Label automatisch die aktuellen Ergebnisse anzeigt.

==== if __name__ == "__main__": Block ====

Arbeitsspeicher:

Die Klasse ModulArbeitsspeicher erbt von tk.Frame und zeigt im GUI alle relevanten RAM- und Swap-Informationen in Echtzeit an. Im Konstruktor wird zunächst ein tk.Label mit Monospace-Schrift (Consolas, 18 pt), linker Ausrichtung und den übergebenen Farben (bg und fg) erzeugt und so gepackt, dass es den gesamten Frame ausfüllt. Anschließend startet die Methode _update_loop(), die in einem vorgegebenen Intervall dynamische Nutzungsdaten abruft und im Label anzeigt.

Die statischen Daten stammen aus der Funktion ausgabe_ram_herst_kapaz_serienn(), die über WMI alle physischen RAM-Steckplätze ausliest und Hersteller, Kapazität (in GB) sowie Seriennummern sammelt und zu einem formatierten Textblock zusammenfügt. Für die dynamischen Werte fragt _update_loop() mit psutil.virtual_memory() den Hauptspeicher (Gesamt, Verfügbar, Benutzt, Frei, Prozentsatz) und mit psutil.swap_memory() den Auslagerungsspeicher (Total, Used, Free, Prozent) ab. Alle Byte-Werte werden zuvor von der Hilfsfunktion einheit_byte() in passende Einheiten (KB, MB, GB …) umgerechnet.

Nach dem Zusammenfügen der Überschriften und Einzelwerte zu einem einzigen String aktualisiert _update_loop() das text-Attribut des Labels und sorgt per self.after(500, self._upda-

Implementierungsphase

te_loop) dafür, dass die Anzeige kontinuierlich erneuert bleibt. So informiert das Modul dauerhaft über Zustand und Auslastung des Arbeitsspeichers und des Swap-Bereichs.

==== if __name__ == "__main__": Block ====

Information:
Die Klasse ModulInformation erbt von tk.Frame und zeigt im GUI eine zentrierte Willkommens- und Info-Nachricht zu Sentinel an. Im Konstruktor wird ein einzelnes tk.Label, zentriertem Text (Anchor „n“), automatischem Zeilenumbruch nach 760 Pixeln und den übergebenen Hintergrund- und Schriftfarben (bg und fg) erzeugt. Abschließend wird das Label per pack(fill="both", expand=True, padx=20, pady=20) platziert.

==== if __name__ == "__main__": Block ====

6 Abnahmephase

Um die Qualität der Projektarbeit zu gewährleisten wurden mehrere Schreibtischtests durchgeführt.
Während der Implementierung musste häufig getestet werden ob die Module und deren Import ins Hauptprogramm korrekt ausgeführt werden.

Außerdem wurde ein Unit-Test im Hauptprogramm durchgeführt, um die Logik hinter dem Frame Management zu prüfen:
Dass in self.frames wirklich alle erwarteten Seiten angelegt werden.
Dass erstelle_frame(page_name) genau das richtige Frame nach oben holt.

Zusätzlich wurde ein Unit-Test im Modul CPU durchgeführt, der die statische Methode ausgabe_cpu_info() mit gemockten WMI-, PSUtil- und Platform-Daten validiert.
Um zu prüfen, ob `update_info()` korrekt Text im ScrolledText-Widget schreibt.

Wichtiger Hinweis: Beide Unit-Tests wurden mit Hilfe von KI generiert!

In der Abnahmephase wurde erfolgreich eine .exe Datei erstellt.

Ein Auszug eines Unit Tests befindet sich im Anhang Error: Reference source not found. Dort ist auch der Aufruf des Tests auf der Konsole des Webservers zu sehen.

7 Einführungsphase

Zur Bereitstellung von Sentinel verwendete ich zunächst den PyInstaller, um die Python-App in eine ausführbare Windows-Anwendung umzuwandeln. Das hat den Vorteil, dass Sentinel auch auf Computern läuft, die keinen Python-Interpreter installiert haben.

Die Software Sentinel kann lokal z. B. auf USB-Sticks an Freunde und Bekannte verteilt werden oder auch über Sharing-Plattformen geteilt werden.
Ein Freund von mir hat es bereits getestet, alle Tests verliefen fehlerfrei (Windows 11).

8 Dokumentation

Eine Benutzerdokumentation wurde erstellt und im Anhang angefügt.

9 Fazit

9.1 Soll-/Ist-Vergleich

Das Projektziel wurde nur teilweise erreicht. Während des Projekts stellte sich heraus das bestimmte Daten nicht ohne Umwege einfach auszulesen sind. Dazu gehören beispielsweise Sensorabfragen, Spannungen und Stromverbrauch. Auch erweiterte Parameter der Grafikkarte ist ohne externe Bibliothek nicht möglich. Das ist allerdings auch Hersteller und Treiber abhängig. Ich habe eine AMD GPU, bei NVIDIA scheint es etwas leichter zu sein. Entweder hätte man über eine .dll Datei oder gar einen Wrapper mit cpp die Daten abrufen könnnen, aber mir fehlte schlicht und einfach die Zeit dazu. Darum konzentrierte ich mich auf grundlegende Daten. Aufgrund des Zeitrahmens entschied ich mich während der Implementierung auch auf die Echtzeitdiagramme, Warnungen, Schwellwertüberwachung und den On-Screen-Display zu verzichten. Auch die Datenbankspeicherung musste ich aus meinem Konzept streichen. Den Zeitaufwand und die enthaltene Recherche wären doch immens hoch. Das habe ich unterschätzt.

Mit meinem Programm bin ich nur bedingt zufrieden, die Grundfunktionen habe ich einbauen können, allerdings ist die Performance während der Laufzeit erschreckend schlecht, das entwickelte sich erst zum Ende des Projekts. Allerdings habe ich mein Programm via SnakeViz auslesen können, allerdings fehlte mir leider die Zeit den Code zu optimieren. Der Fehler scheint mit den vielen WMI-Abfragen zusammenzuhängen. Somit läuft das Programm nur suboptimal.

Wie in der folgenden Tabelle zu sehen ergaben sich einige Abweichungen von der geplanten Durchführung.

Phase	Geplant	Tatsächlich	Differenz
Analyse	3h	4h	+1h
Entwurf	3h	5h	+2h
Implementierung	35h	40h	+5h
Abnahme	2h	1h	-1h
Einführung	2h	1h	-1h
Dokumentation	35h	30h	-5h
Gesamt	**80h**	**81h**	**+1h**

Tabelle : Soll-/Ist-Vergleich

9.2 Lessons Learned

Während der Umsetzung meines Projekts habe ich unglaublich viel lernen können. Mir ist sehr schnell aufgefallen das eine effiziente Entwicklung von Software, viel Planung voraussetzt. Natürlich möchte man schnell programmieren und sich ausprobieren, aber ohne zumindest grobe Struktur merkt man dass man sich eventuell übernimmt und zu viel Zeit in Gedankenumsetzungen verbringt.

Fazit

So wollte ich z. B. unbedingt Temperaturen etc. der Grafikkarte auslesen und habe etwas zu viel Zeit damit verbracht. Am Ende musste ich mich auf das Wesentliche beschränken, damit ich meine anderen Module fertigstellen konnte.
So erging es mir allgemein mit meinen Ideen. Ich hatte und habe immer noch einige (siehe Ausblick),von denen ich auch schon manche von Anfang an geplant hatte, doch der nicht geahnte Umfang holte mich schnell ein. So habe ich viele Abstriche machen müssen.
Beim Programmieren selbst habe ich gelernt, das eine übersichtliche Struktur sehr viel effizienter ist. Anfangs hatte ich Klassen und deren Funktionen alle im Hauptprogramm, bis ich mehrmals den Überblick verlor und mich dazu entschloss zu modularisieren. So konnte ich getrennt und unabhängig meine Module testen und ins Hauptprogramm importieren.
Allerdings bin ich auf einige Performance Probleme gestoßen, die noch nicht gänzlich behoben sind. Das weiß ich in Zukunft besser zu bedenken.
Ich blicke nach diesem Projekt realistischer auf die Umsetzung einer Software Entwicklung und kann mit Sicherheit sagen dass ich viel gelernt habe, aber auch sehr viel Freude an der Umsetzung hatte.

9.3 Ausblick

Mein Projekt ist für mich noch lange nicht ausgereift, deshalb sind noch viele Erweiterungen und Verbesserungen geplant.
Primär gilt es erst mal die Performance zu optimieren. Diese ist doch viel zu träge und wird mit kommenden, neuen Modulen und Klassen sicher nicht effizienter, wenn der Code nicht optimiert wird.
Nach erfolgreicher Optimierung möchte ich neue Ideen und Funktionen implementieren:

- Temperaturabfragen über vorhandene Sensoren
- Schwellwertüberwachung mit Warnungen (Desktop Pop-Up, eventuell Benachrichtigung via Telegram-Bot)
- Spannung und Stromverbrauch der jeweiligen Hardware (wie auch Lüfter etc.)
- Benchmarkings um die Leistung des Systems zu testen (eventuell mit Upload auf einer Vergleichsseite für z.B. CPU Rankings)
- Visualisierung der Auslastungen mit Echtzeitdiagrammen
- Datenbankspeicherung für eine Langzeitanalyse der Gesamt-Performance, Visualisierung mit Langzeitdiagrammen.
- Eine überarbeitete Benutzeroberfläche
- Lüfter-Überwachung

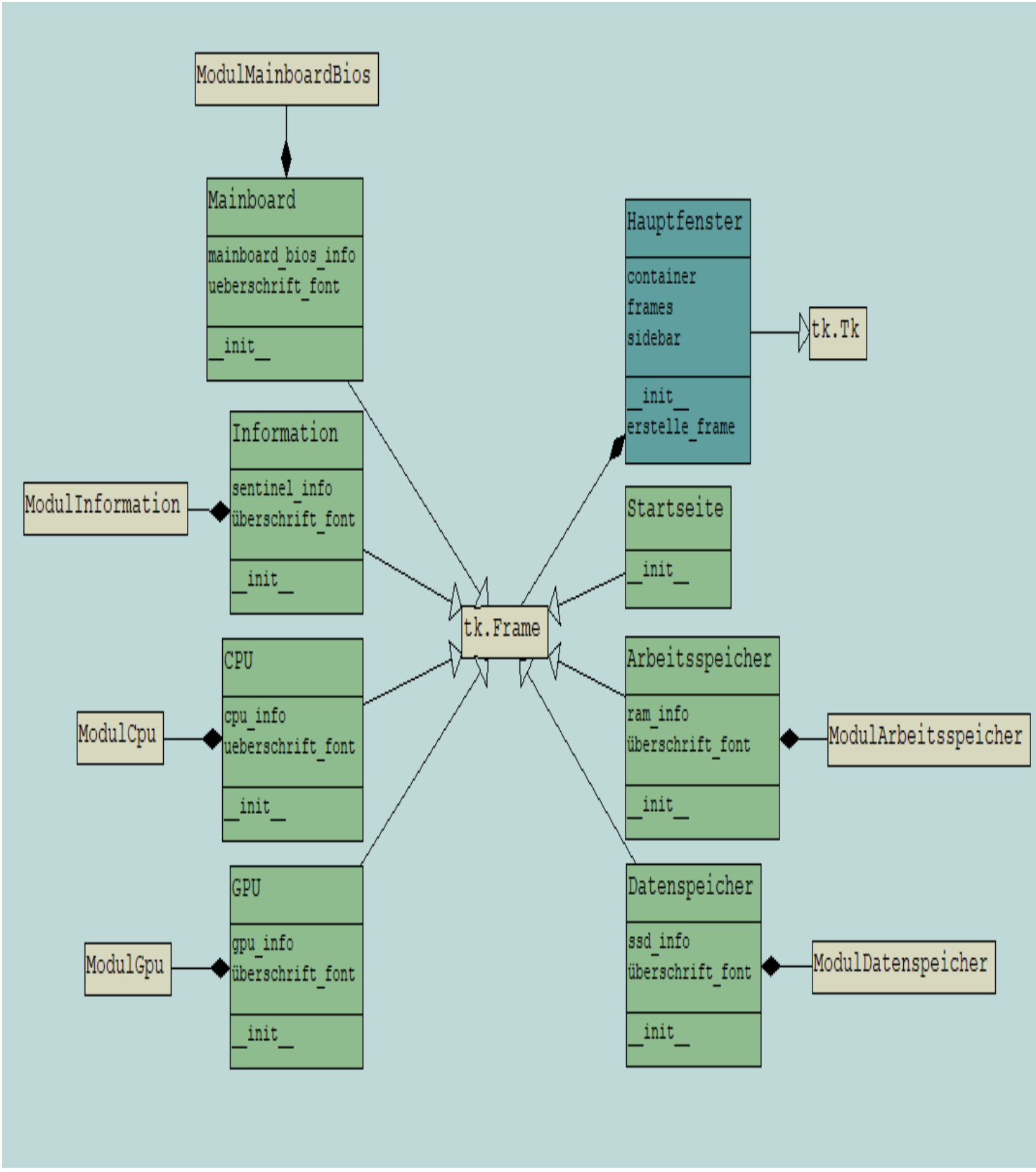
Die Zukunft wird noch viele gute Ideen bringen, die es umzusetzen gilt. Auf dieser Basis werde ich aufbauen und freue mich auf das Kommende.

Anhang

Quellenverzeichnis

1. Handbuch für Softwareentwickler von Krypczyk Bochkor (Rheinwerk)
2. IT-Handbuch für Fachinformatiker*innen von Sascha Kersken (Rheinwerk)
3. Einstieg in Python von Thomas Theis (Rheinwerk)
4. https://www.python.org
5. https://docs.python.org/3/library/tkinter.ttk.html#using-ttk
6. https://cs111.wellesley.edu/archive/cs111_fall14/public_html/labs/lab12/tkintercolor.html
7. https://www.python-kurs.eu/python3_modularisierung.php
8. https://www.python-kurs.eu/python3_bloecke.php
9.https://www.reddit.com/r/Python/comments/1k8glge/can_ai_play_a_role_in_creating_automated_software

Abbildung: Klassendiagramm vom Sentinel Hauptfenster:



SENTINEL HARDWARE MONITOR
SOFTWARE ZUR ÜBERWACHUNG VON HARDWARE-DATEN

Anhang

Benutzerdokumentation:

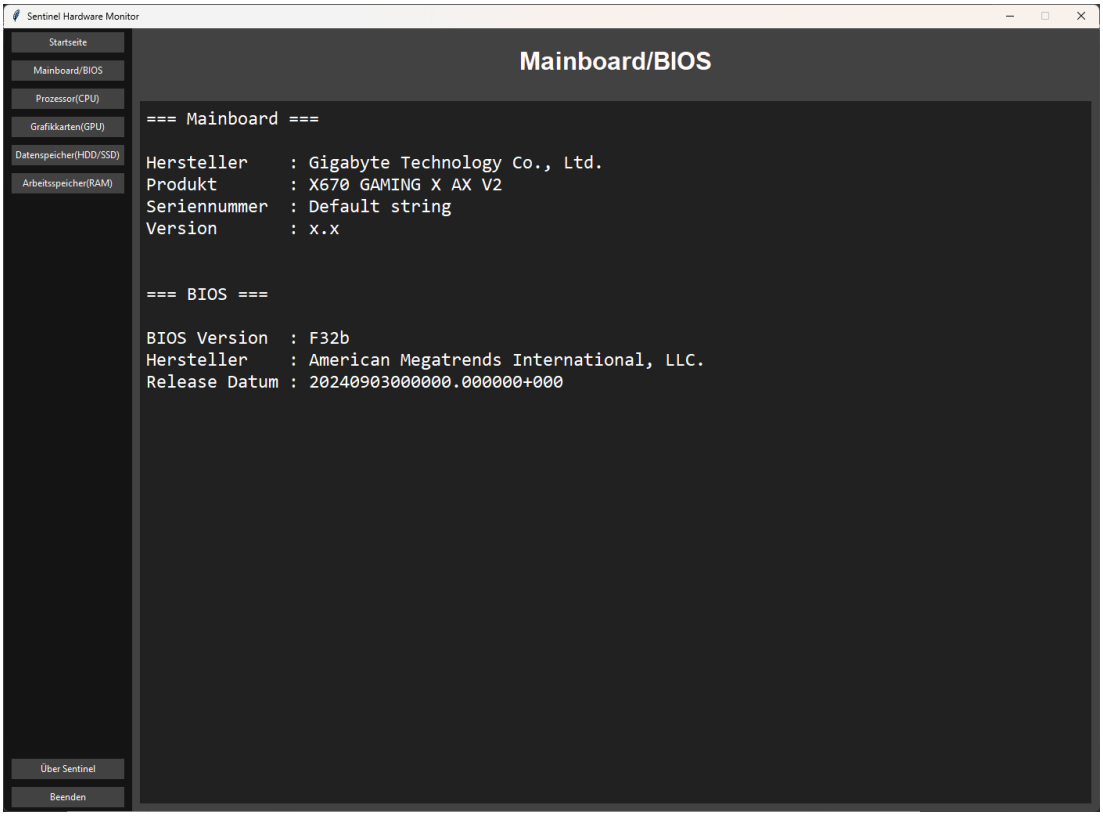
Wenn Sie die .exe starten sehen Sie die Startseite mit Logo.

Um andere Menüpunkte aufzurufen klicken Sie den jeweiligen Button links in der Sidebar.

Nachfolgend sehen Sie alle Frames (von oben nach unten).



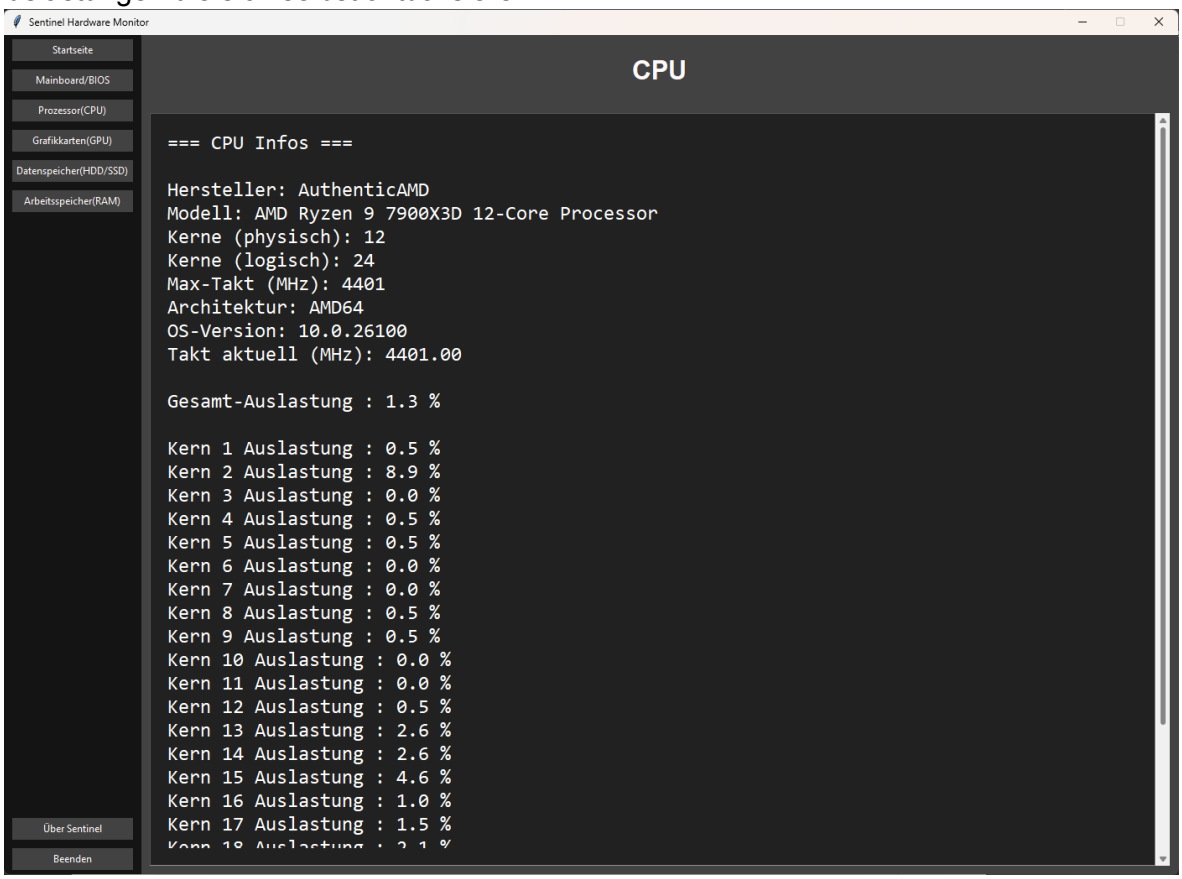
Hier sehen Sie die Mainboard/BIOS Informationen



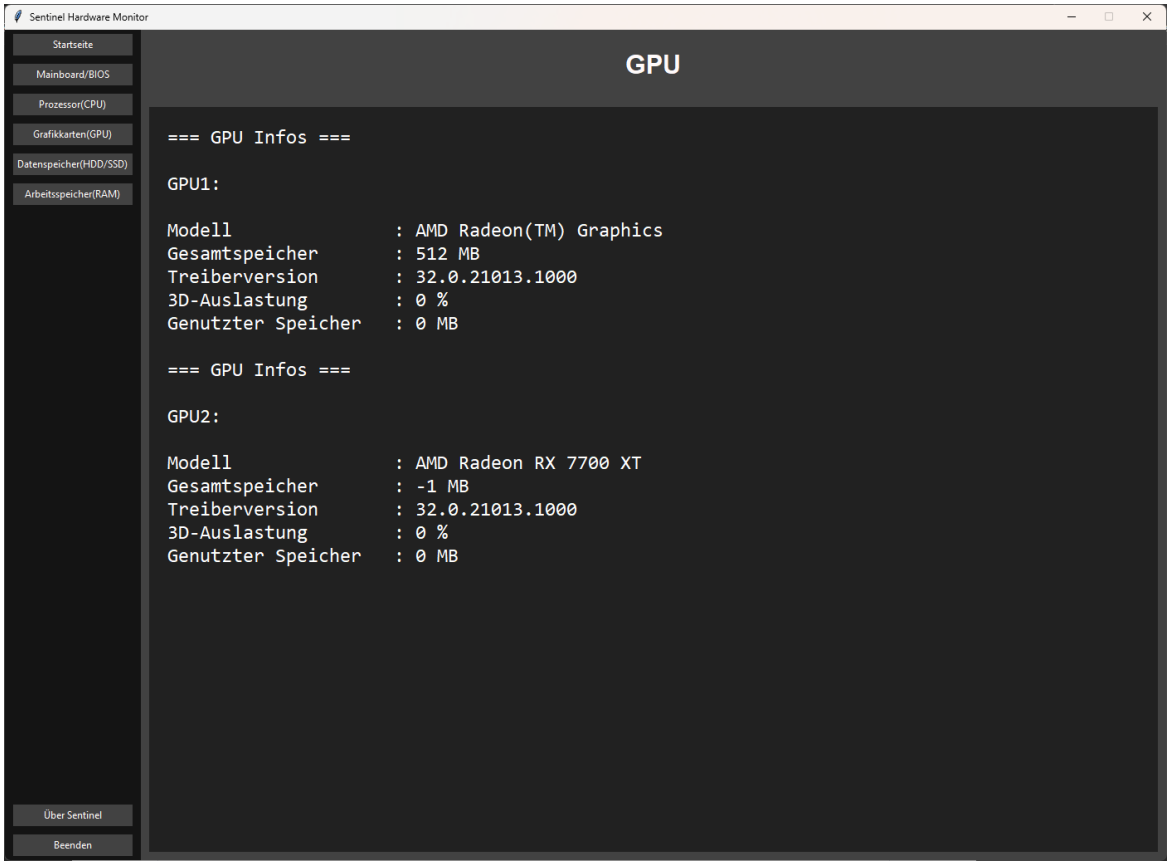
SENTINEL HARDWARE MONITOR
SOFTWARE ZUR ÜBERWACHUNG VON HARDWARE-DATEN

Anhang

Hier sehen Sie zusätzlich zu den statischen CPU Informationen auch die dynamischen Kernauslastungen die sich selbst aktualisieren.



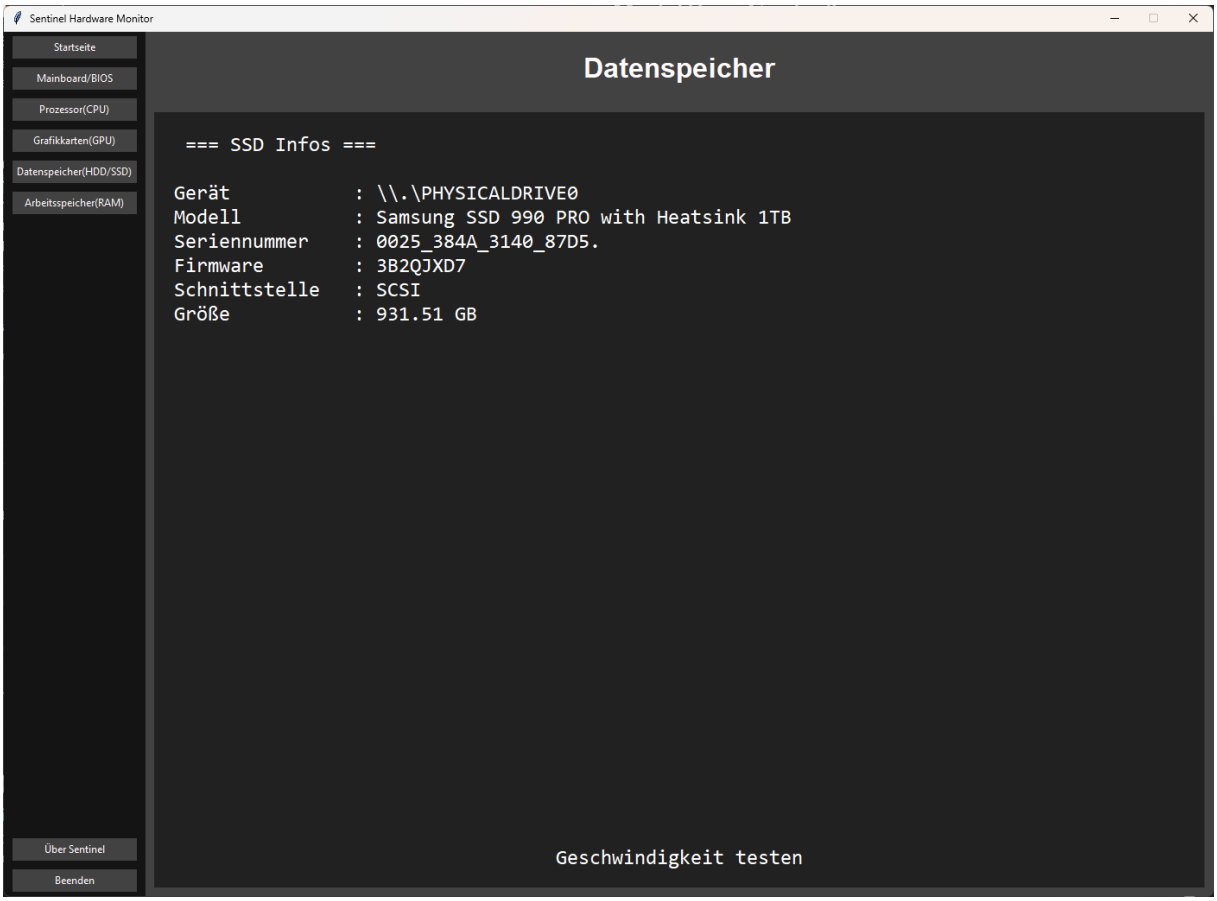
Hier sehen Sie die GPU Informationen.



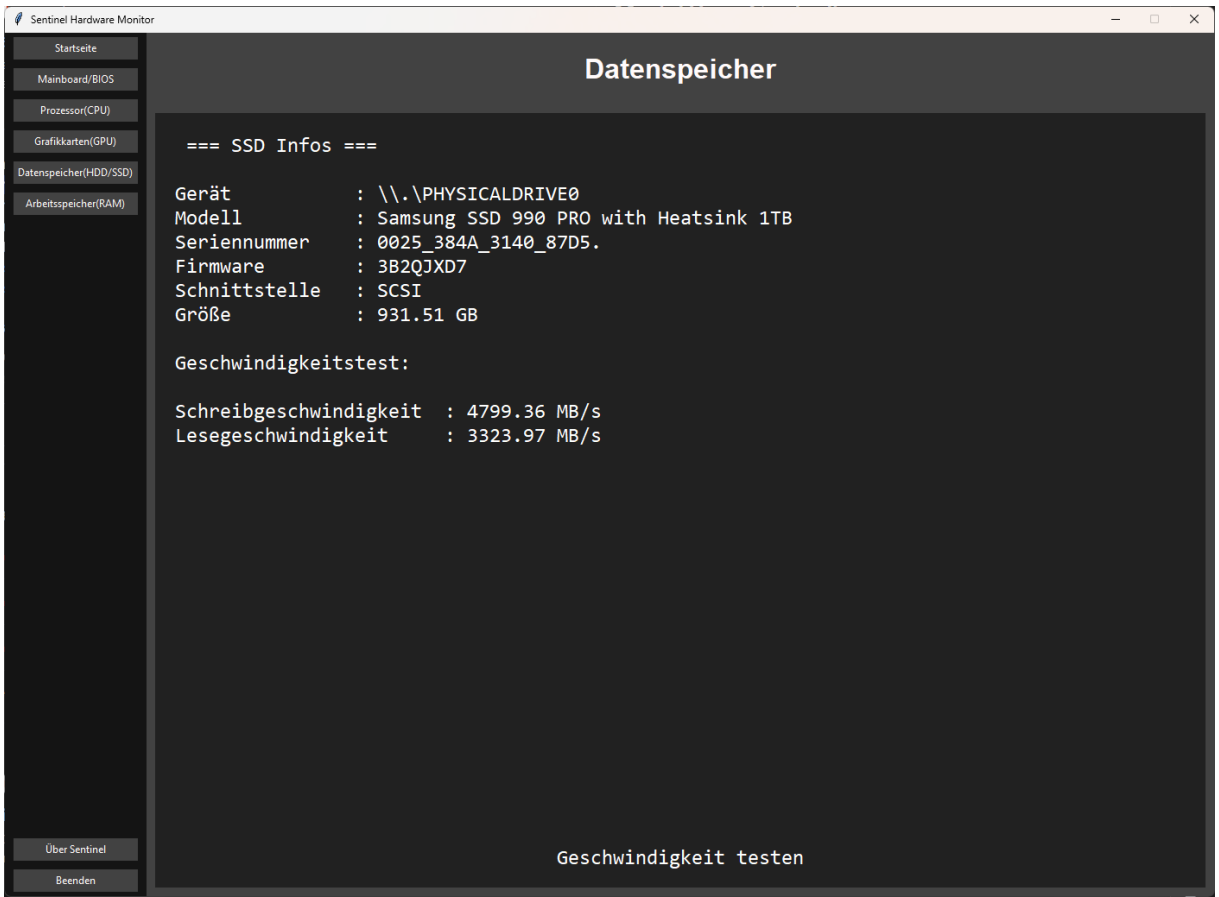
SENTINEL HARDWARE MONITOR
SOFTWARE ZUR ÜBERWACHUNG VON HARDWARE-DATEN

Anhang

Hier sehen Sie die SSD Informationen.



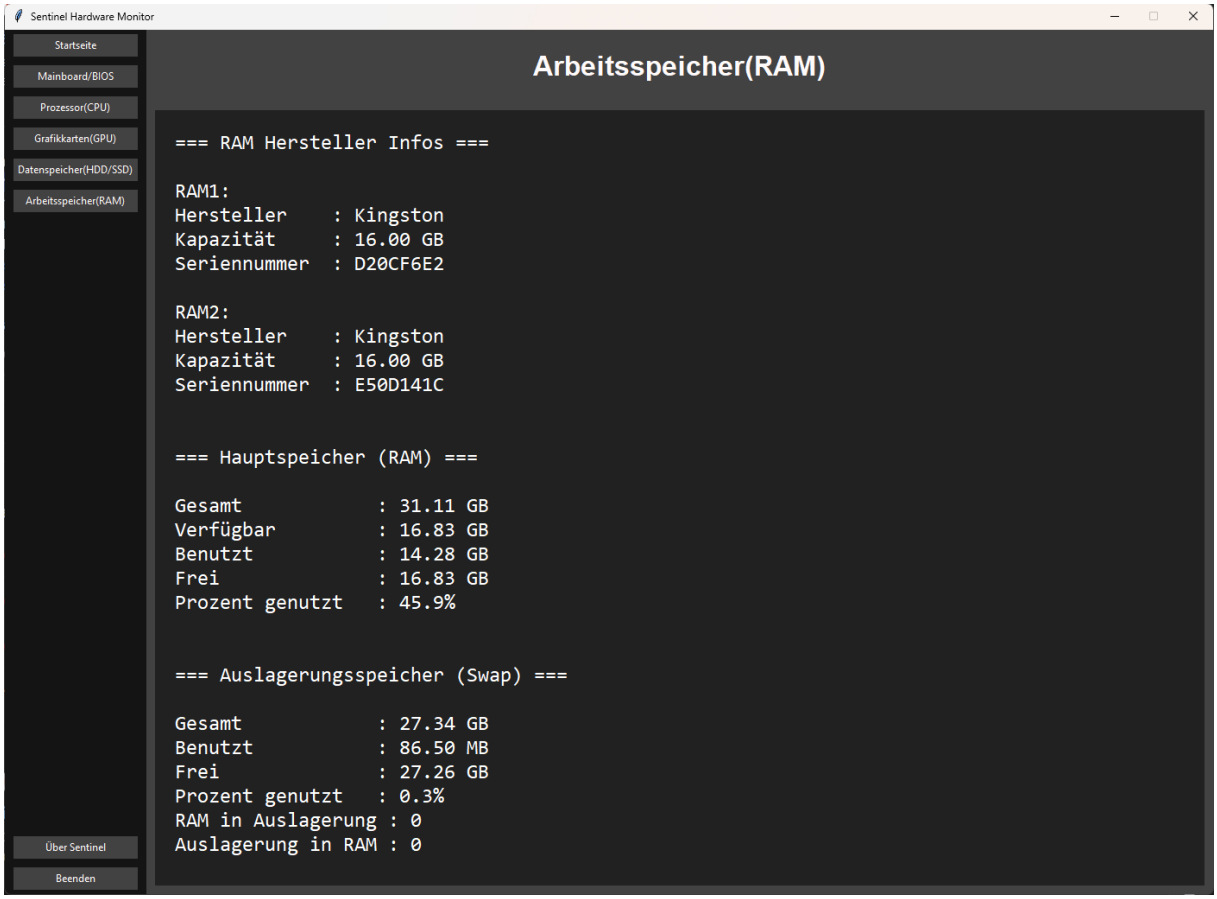
Wenn Sie den Button Geschwindigkeit testen drücken wird die lese- und Schreibgeschwindigkeit getestet. Die Ausgabe erfolgt unmittelbar. Wenn Sie erneut drücken werden die alten Werte mit neuen überschrieben.



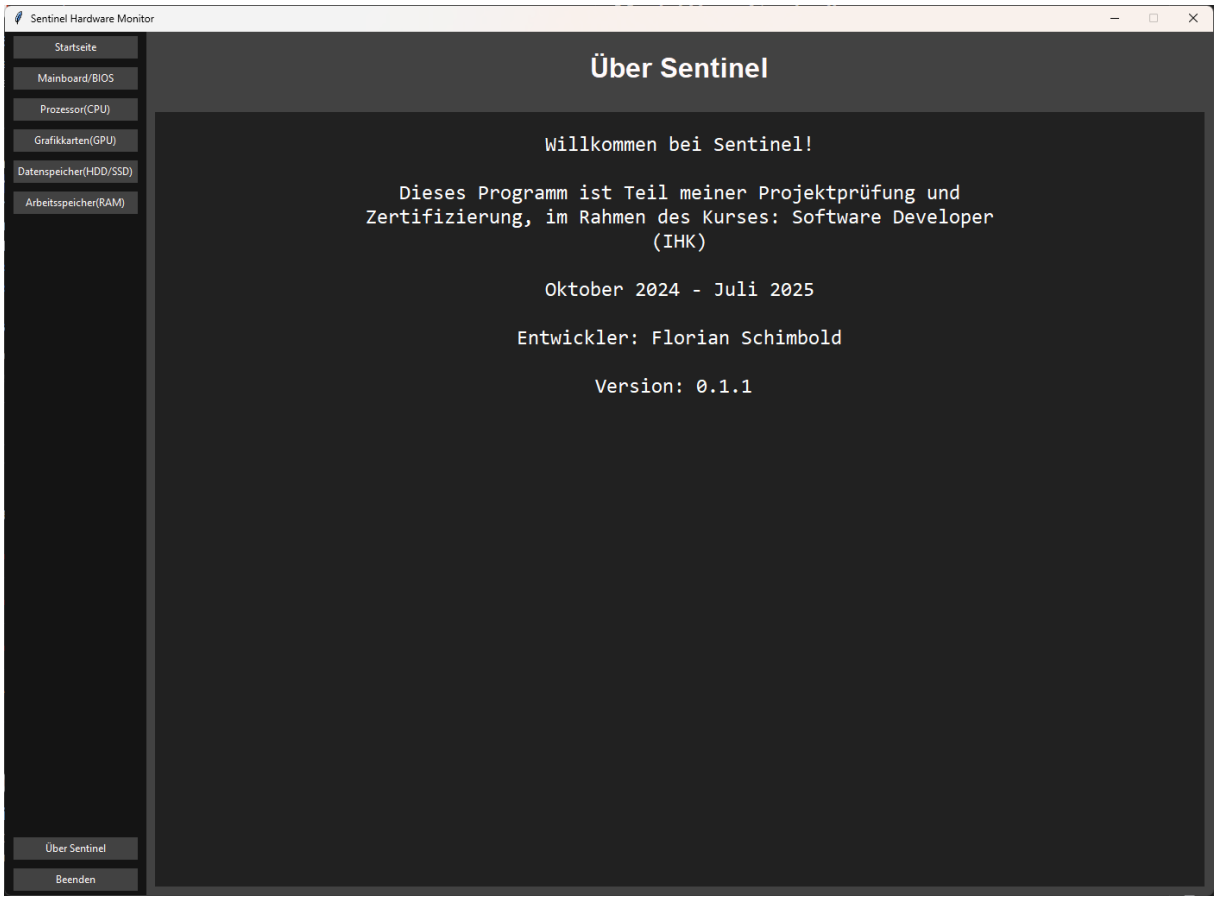
SENTINEL HARDWARE MONITOR
SOFTWARE ZUR ÜBERWACHUNG VON HARDWARE-DATEN

Anhang

Hier sehen Sie die Arbeitsspeicher Informationen.



Hier sehen Sie allgemeine Informationen. Zum beenden den Button Beenden drücken.



SENTINEL HARDWARE MONITOR
SOFTWARE ZUR ÜBERWACHUNG VON HARDWARE-DATEN

Anhang

Code aus dem Hauptprogramm:

sentinel_hardware_monitor - sentinel_hardware_monitor_final.py

```python
# --------------------------------
# Sentinel Hardware Monitor / Projektarbeit zur Zertifizierung / Software Developer (IHK) 2024-2025
# --------------------------------
# Florian Schimbold
#---------------------------------

import tkinter as tk
from PIL import Image, ImageTk
import tkinter.font as tkfont

from sentinel_datenspeicher_info import ModulDatenspeicher
from sentinel_mainboard_bios_info import ModulMainboardBios
from sentinel_ram_info import ModulArbeitsspeicher
from sentinel_cpu_info import ModulCpu
from sentinel_gpu_info import ModulGpu
from ueber_sentinel import ModulInformation


class Hauptfenster(tk.Tk):  3 usages
    def __init__(self):
        super().__init__()
        self.title("Sentinel Hardware Monitor")
        self.configure(bg="gray26")
        self.geometry("1400x1000")
        self.resizable(width=0, height=0)

        farbe_hintergrund_menue = "gray8"
        farbe_hintergrund_container = "gray26"
        farbe_hintergrund_button = "gray26"
        farbe_button_klick = "gray43"
        farbe_schrift = "snow"
        # Sidebar zur Navigation
        self.sidebar = tk.Frame(self, width=800, bg=farbe_hintergrund_menue)
        self.sidebar.pack(side="left", fill="y")
        # Container für wechselnde Frames
        self.container = tk.Frame(self, bg=farbe_hintergrund_container)
        self.container.pack(side="right", fill="both", expand=True)
        # Dictionary um alle Frames zu speichern
        self.frames = {}
        for fenster in (Startseite, Mainboard, CPU, GPU, Datenspeicher, Arbeitsspeicher, Information):
            seiten_name = fenster.__name__
            frame = fenster(parent=self.container,
                            controller=self,
                            bg=farbe_hintergrund_container,
                            fg=farbe_schrift)
```

Anhang

Code aus dem Hauptprogramm:



SENTINEL HARDWARE MONITOR
SOFTWARE ZUR ÜBERWACHUNG VON HARDWARE-DATEN

Anhang

Code aus dem Hauptprogramm:

sentinel_hardware_monitor - sentinel_hardware_monitor_final.py

```python
class Hauptfenster(tk.Tk):  3 usages
    def __init__(self):
        # Standard Frame
        self.erstelle_frame("Startseite")

    def erstelle_frame(self, page_name):  5 usages
        frame = self.frames[page_name]
        frame.tkraise()
# Klassen der jeweiligen Module die importiert werden
class Startseite(tk.Frame):  1 usage
    def __init__(self, parent, controller, bg, fg):
        super().__init__(parent, bg=bg)
        sentinel_logo = Image.open(r"C:\Users\fschi\PycharmProjects\sentinel_hardware_monitor\Sentinel_Logo_v2.jpg")
        bild_logo = ImageTk.PhotoImage(sentinel_logo)
        logo_label = tk.Label(self, image=bild_logo)
        logo_label.image = bild_logo
        logo_label.pack()

class Mainboard(tk.Frame):  1 usage
    def __init__(self, parent, controller, bg, fg):
        super().__init__(parent, bg=bg)
        self.ueberschrift_font = tk.font.Font(family="Helvetica", size=24, weight="bold")
        frame_mainboard = tk.Label(self, text="Mainboard/BIOS", bg=bg, fg=fg, font=self.ueberschrift_font)
        frame_mainboard.pack(pady=20, padx=20)
        self.mainboard_bios_info = ModulMainboardBios(self)
        self.mainboard_bios_info.pack(fill="both", expand=True, padx=10, pady=10)

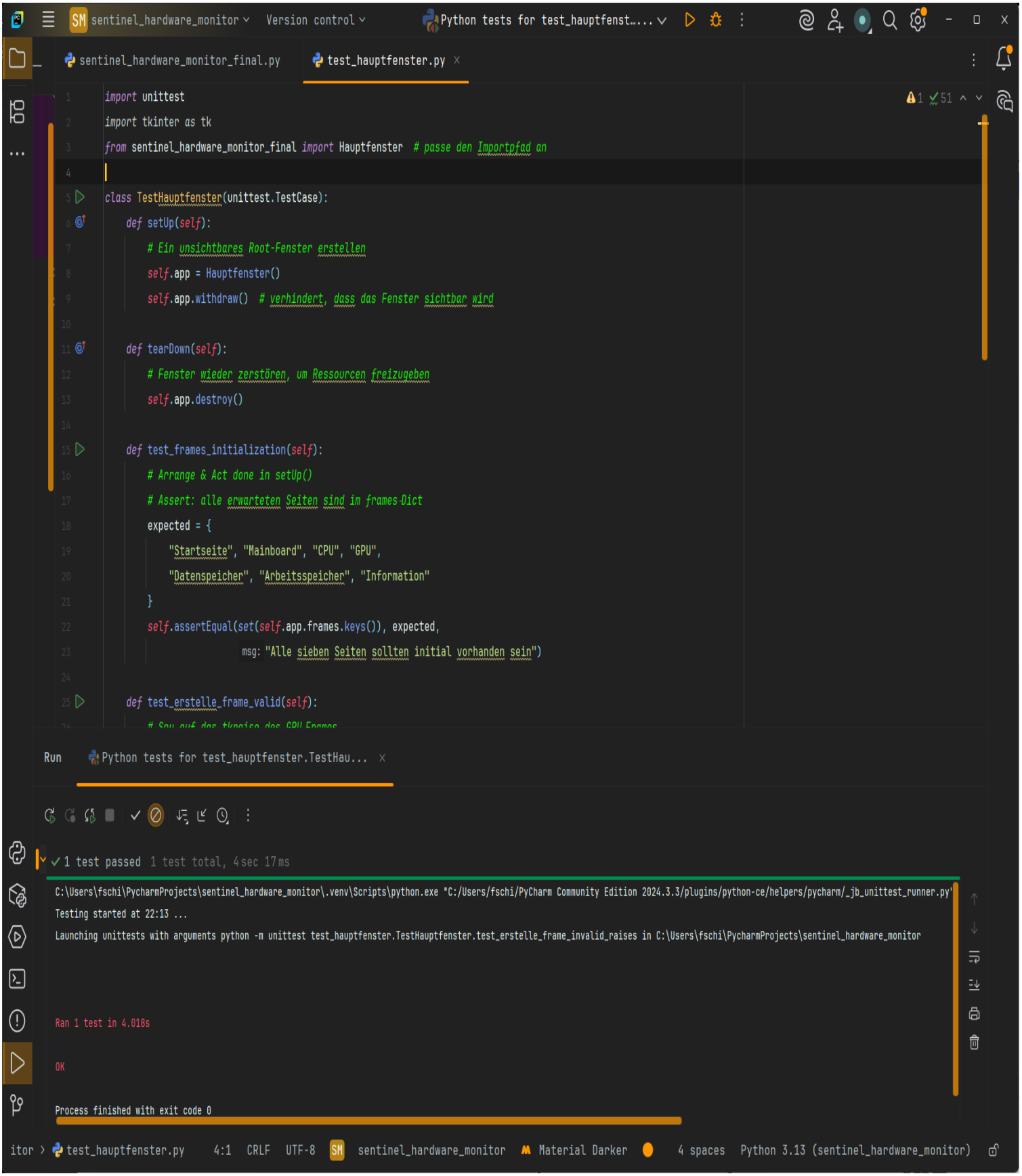
class CPU(tk.Frame):  1 usage
    def __init__(self, parent, controller, bg, fg):
        super().__init__(parent, bg=bg)
        self.ueberschrift_font = tk.font.Font(family="Helvetica", size=24, weight="bold")
        frame_cpu = tk.Label(self, text="CPU", bg=bg, fg=fg, font=self.ueberschrift_font)
        frame_cpu.pack(pady=20, padx=20)
        self.cpu_info = ModulCpu(self, relief=tk.FLAT)
        self.cpu_info.pack(fill="both", expand=True, padx=10, pady=10)

class GPU(tk.Frame):  1 usage
    def __init__(self, parent, controller, bg, fg):
        super().__init__(parent, bg=bg)
        self.überschrift_font = tk.font.Font(family="Helvetica", size=24, weight="bold")
        frame_gpu = tk.Label(self, text="GPU", bg=bg, fg=fg, font=self.überschrift_font)
        frame_gpu.pack(pady=20, padx=20)
        self.gpu_info = ModulGpu(self, relief=tk.FLAT)
        self.gpu_info.pack(fill="both", expand=True, padx=10, pady=10)
```

SENTINEL HARDWARE MONITOR
SOFTWARE ZUR ÜBERWACHUNG VON HARDWARE-DATEN

Anhang

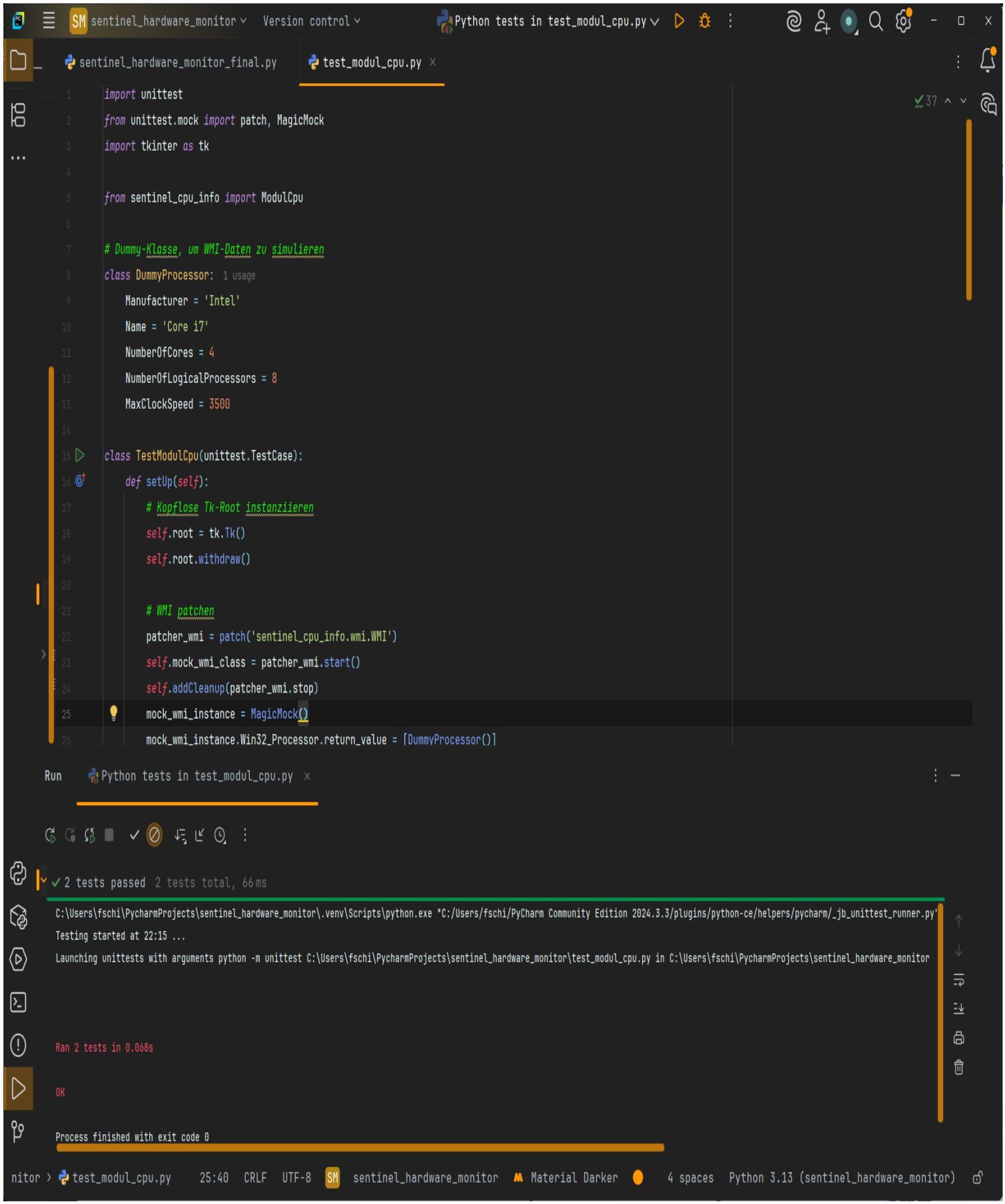
Unit-Test für das Hauptprogramm:



SENTINEL HARDWARE MONITOR
SOFTWARE ZUR ÜBERWACHUNG VON HARDWARE-DATEN

Anhang

Unit-Test für das Modul CPU:



Anhang

Beispiel MVC Pattern als ASCII-Diagramm:

